

Chapter 6 — Redirection

The Linux Command Line — Study Notes

Redirection is the mechanism for changing *where a command's input comes from* and *where its output goes*. By default input arrives from the keyboard and output goes to the screen, but with redirection those endpoints can be rewired to files — or, via pipes, chained directly into other commands to form powerful pipelines.

1 Standard Streams

Every command is wired to three numbered channels, called **file descriptors**. This is the foundation everything else in the chapter builds on.

The Three Standard Streams

- **Standard input** (`stdin`, descriptor **0**) — where input comes from; by default, the keyboard.
- **Standard output** (`stdout`, descriptor **1**) — where a program's results go; by default, the screen.
- **Standard error** (`stderr`, descriptor **2**) — where status and error messages go; by default, also the screen.

A key insight: a program's *results* and its *error messages* travel on two *separate* channels. This is why redirecting output to a file can still leave an error message printed on screen — the error rode on `stderr`, not `stdout`. It follows the Unix theme that “everything is a file”: even the screen and keyboard are treated as files.

2 Redirecting Standard Output: > and »

The > operator sends `stdout` to a file instead of the screen:

```
ls -l /usr/bin > ls-output.txt
```

> Overwrites — Silently and Ruthlessly

The destination file is **always rewritten from the beginning**. If the command produces no output (e.g. it fails with an error), the file is truncated to **zero length**. The > operator silently creates or overwrites files, so treat it with respect — `ls > less` inside `/usr/bin` would overwrite the `less` program itself.

To **append** rather than overwrite, use ». It adds to the end of the file, creating it first if it does not exist:

```
ls -l /usr/bin >> ls-output.txt
```

Truncation Trick

A redirection operator with *no command in front of it* truncates an existing file or creates a new empty one:

```
> ls-output.txt
```

2.1 Group Commands

To send the combined output of several commands to one file, wrap them in braces and redirect once. The shell then treats the group as a single stream:

```
{ command1; command2; command3; } > logfile.txt
```

The shell requires whitespace around the braces, and the final command must end with a semicolon or newline.

3 Redirecting Standard Error

Standard error has no dedicated operator; you must reference its file descriptor number, **2**, placed immediately before the operator:

```
ls -l /bin/usr 2> ls-error.txt
```

3.1 Combining stdout and stderr into One File

Traditional method — redirect stdout to the file, then point descriptor 2 at descriptor 1:

```
ls -l /bin/usr > ls-output.txt 2>&1
```

Order Is Significant

The redirection of standard error must come *after* the redirection of standard output. Reversed (`2>&1 > file`), stderr still goes to the screen, because `2>&1` copies wherever descriptor 1 pointed *at that moment* — which is still the screen until the `>` runs.

Modern method — a single notation for both streams:

```
ls -l /bin/usr &> ls-output.txt      # overwrite both
ls -l /bin/usr &>> ls-output.txt     # append both
```

3.2 Disposing of Unwanted Output

Redirect to `/dev/null` — the “bit bucket,” a system device that accepts input and discards it — to throw output away entirely:

```
ls -l /bin/usr 2> /dev/null
```

4 Redirecting Standard Input: <

The `cat` command reads files and copies them to `stdout`:

```
cat lazy_dog.txt
```

With **no arguments**, `cat` reads from `stdin` (the keyboard) and echoes it back — press `Ctrl-d` to signal end-of-file. This lets you create a quick text file:

```
cat > lazy_dog.txt      # type text, then Ctrl-d
```

The `<` operator changes the source of `stdin` from the keyboard to a file:

```
cat < lazy_dog.txt
```

Filename Argument vs. Redirected Input

When a command is *given a filename argument*, the command opens the file itself and knows its name. When a file is fed in via `<` (or a pipe), the command reads a nameless `stdin` stream and cannot see any filename. This is why `wc -l file` prints the name but `wc -l < file` prints only the number.

5 Pipelines: |

The pipe operator connects the `stdout` of one command directly to the `stdin` of the next — no file involved, data flows in memory:

```
command1 | command2  
ls -l /usr/bin | less
```

The Difference Between `>` and `|`

`>` connects a command to a **file**.

`|` connects the output of one command to the **input of another command**.

Never write `command1 > command2` expecting a pipe — that overwrites a *file* named `command2` with the output of `command1`.

6 Filters

Commands chained in a pipeline that take input, transform it, and pass it on are called **filters**. The chapter introduces these workhorses.

6.1 sort — Sort Lines of Text

Rearranges lines into sorted order. Essential as a preprocessing step because many tools only work on adjacent lines.

```
ls /bin /usr/bin | sort | less
```

6.2 `uniq` — Report or Omit Repeated Lines

Removes duplicate lines — but **only when they are adjacent**. It compares each line to the one immediately before it, so input almost always needs `sort` first.

```
ls /bin /usr/bin | sort | uniq      # remove duplicates
ls /bin /usr/bin | sort | uniq -d   # show only duplicates
```

The `-c` option prefixes each line with a count of occurrences.

6.3 `wc` — Line, Word, and Byte Counts

Prints lines, words, and bytes. With no arguments it reads `stdin`. The `-l` option limits output to a line count — handy at the end of a pipeline for counting things.

```
ls /bin /usr/bin | sort | uniq | wc -l
```

6.4 `grep` — Print Lines Matching a Pattern

Prints every line containing a pattern: `grep pattern [file...]`. Useful options:

- `-i` — ignore case
- `-l` — print only the names of matching files
- `-v` — print only lines that do *not* match
- `-w` — match only whole words

```
ls /bin /usr/bin | sort | uniq | grep zip
```

Patterns Are Regular Expressions, Not Globbs

`grep`'s pattern is a *regular expression*, interpreted by `grep` itself — not a shell wildcard. In regex, `*` means “zero or more of the preceding character,” so `grep ERROR*` means “ERRO followed by any number of R’s,” not “lines containing ERROR.” For a plain substring match, just write `grep ERROR`.

6.5 `head` / `tail` — First / Last Part of Files

`head` prints the first 10 lines, `tail` the last 10; adjust with `-n`.

```
head -n 5 ls-output.txt
tail -n 5 ls-output.txt
```

Advanced forms let you cut from the middle: `head -n -5` prints all but the last 5 lines; `tail -n +6` prints from line 6 onward. `tail -f` follows a file in real time — ideal for watching log files grow (`Ctrl-c` to stop).

6.6 tee — Split the Stream

`tee` reads `stdin` and copies it to *both* `stdout` (so the pipeline continues) *and* to one or more files. It is a “tee fitting” on the pipe, used to capture a pipeline at an intermediate stage:

```
ls /usr/bin | tee ls.txt | grep zip
```

7 Summary

Operators at a Glance

>	redirect stdout to a file (overwrite)
»	redirect stdout to a file (append)
<	redirect a file into stdin
2>	redirect stderr to a file
2>&1	redirect stderr to wherever stdout points
&>	redirect both stdout and stderr (overwrite)
&»	redirect both stdout and stderr (append)
	pipe stdout of one command into stdin of the next

The deeper lesson is the Unix design philosophy: because every program speaks the same simple language — read bytes from `stdin`, write bytes to `stdout` — small single-purpose tools can be snapped together in endless combinations. A command cannot tell whether its input came from the keyboard, a file, or another program; to it, input is just a stream. That uniformity is what makes pipelines possible, and it is why the command line “takes on the shape of your imagination.”